

Module 7 — Automation

Lesson 7.2

Claude Code in GitHub Actions

Install the CLI in your runner, set the API key as a secret, run `claude -p` as a step. Same patterns extend to any CI.

Mastering Claude Code — An E-Course for Power Users

Why it matters

Putting Claude in CI moves toil off engineers. New issues get auto-triaged. Failing builds get diagnosed. Stale PRs get summarized. Routine reviews happen before a human ever looks. None of this is magic — it's `claude -p` wired into a workflow.

This lesson walks through `examples/06-github-action`, which auto-triages new issues by suggesting labels.

The pattern

```
GitHub event ■■■ workflow trigger ■■■ install claude ■■■ run `claude -p ...` ■■■ post result  
via gh CLI
```

Five steps. Most of the complexity is in writing a good prompt.

Minimal workflow

```

name: Triage new issues

on:
  issues:
    types: [opened]

jobs:
  triage:
    runs-on: ubuntu-latest
    permissions:
      issues: write
      contents: read
    steps:
      - uses: actions/checkout@v4

      - name: Install Claude Code
        run: curl -fsSL https://claude.ai/install.sh | bash
        # Or: npm install -g @anthropic-ai/claude-code

      - name: Triage issue
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
          GH_TOKEN: ${ secrets.GITHUB_TOKEN }
          ISSUE_NUMBER: ${ github.event.issue.number }
          ISSUE_TITLE: ${ github.event.issue.title }
          ISSUE_BODY: ${ github.event.issue.body }
        run: |
          PROMPT="$(cat <<EOF
          Triage this GitHub issue. Suggest 1-3 labels from this set:
          bug, feature, docs, question, dup.

          Title: $ISSUE_TITLE
          Body: $ISSUE_BODY

          Reply as a JSON object: {"labels": ["..."], "reason": "..."}
          EOF
          )"

          RESULT=$(echo "$PROMPT" | claude -p --output-format json
          --dangerously-skip-permissions)
          LABELS=$(echo "$RESULT" | jq -r '.messages[-1].content[0].text' | jq -r '.labels[]'
          | tr '\n' ',' | sed 's/,$/'/)

          if [ -n "$LABELS" ]; then
            gh issue edit "$ISSUE_NUMBER" --add-label "$LABELS"
          fi

```

Walking through:

- Triggered when an issue is opened.
- `permissions:` grants `issues: write` to the workflow's `GITHUB_TOKEN`.
- Install Claude Code into the runner.

- Build a prompt from the issue title/body.
- Pipe into `claude -p` with JSON output.
- Parse the labels out with `jq`.
- Apply them via `gh issue edit`.

`--dangerously-skip-permissions` is appropriate here: the runner is ephemeral, the task is bounded, and there's no human to prompt.

Secrets

You need at minimum:

- `ANTHROPIC_API_KEY` — set as a GitHub repo (or org) secret. Used by Claude Code for auth in CI (subscription auth is for humans only).

If your action also calls GitHub APIs, the built-in `GITHUB_TOKEN` (or a fine-grained PAT) is usually sufficient. Don't expose API keys via `echo` in logs.

Useful workflow patterns

PR diff review

```
on: pull_request
steps:
  - uses: actions/checkout@v4
    with: { fetch-depth: 0 }
  - run: curl -fsSL https://claude.ai/install.sh | bash
  - env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
    run: |
      DIFF=$(git diff origin/${ github.base_ref }...HEAD)
      echo "$DIFF" | claude -p "Review for correctness and security. List top 3 issues with
      file:line. Reply as markdown." \
        --dangerously-skip-permissions > review.md
      gh pr comment ${ github.event.pull_request.number } --body-file review.md
    env:
      GH_TOKEN: ${ secrets.GITHUB_TOKEN }
```

Stale issue summarizer (cron)

```
on:
  schedule:
    - cron: '0 14 * * MON' # every Monday at 14:00 UTC
```

Then list stale issues with `gh`, pipe into Claude, post the summary as a discussion.

Auto-fix failing lint on PR

```
on: pull_request
steps:
  - uses: actions/checkout@v4
  - run: pnpm install && pnpm lint --fix-dry-run > lint.txt || true
  - env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
    run: |
      cat lint.txt | claude -p --permission-mode acceptEdits \
        "Fix every issue listed. Don't change behavior. Stage with git add but don't commit." \
        --dangerously-skip-permissions
      git config user.email "claude@bot.local"
      git config user.name "claude-bot"
      git diff --quiet || git commit -m "fix: lint"
      git push
```

(In practice you'd want guards: only on certain branches, only by certain authors, etc. Don't auto-commit on protected branches.)

Cost considerations

- A triage workflow on every new issue might cost cents per issue. For 100 issues/day, that's \$1-10/day.
- A diff-review workflow on every PR can run wide — set sensible budgets per repo.
- For large repos, scope what Claude reads: don't pipe the entire repo, just the diff or just the issue.

Safety considerations

- Workflows triggered by PR comments or issues are public-input territory. Be careful: a malicious user can craft an issue body that tries to prompt-inject Claude. Mitigations:
- Treat content as data (delimit, instruct)
- Don't give Claude the ability to merge, close, or delete from external-trigger workflows
- Use `pull_request_target` carefully — it runs with write tokens
- Don't run untrusted forks' Claude actions with secrets. `pull_request` from forks is safer (no secrets, no write token) than `pull_request_target`.

Try it

- 1 Copy `examples/06-github-action/.github/workflows/claude-triage.yml` into a test repo.
- 2 Add the `ANTHROPIC_API_KEY` secret.
- 3 Open an issue. Watch the action run. Confirm labels appear.
- 4 Tune the prompt to your label vocabulary.

Gotchas

- **Cold install per run** — installing Claude Code in every run adds ~30s. Consider caching the binary or using a container with it pre-installed.
- **Concurrency** — high-volume repos can fire many workflows at once. Stay within Anthropic rate limits.
- **API key in logs** — don't `echo $ANTHROPIC_API_KEY` ever. GitHub Actions masks secrets in logs but not perfectly in all subprocesses.
- **Failed runs cost money too** — a workflow that errors out after Claude returns has still incurred the model cost. Fail fast.

Further reading

- `examples/06-github-action` — runnable
- 7.3 Scheduled and event-driven agents
- Official Claude Code GitHub action discussion:
<https://docs.claude.com/en/docs/claude-code/github-actions>

Next

→ 7.3 Scheduled and event-driven agents